FIT2109 Computer Science Workshop

Week 2: Shell Tools & Scripting

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

From Week 1 to reliable automation

Week 1 gave us the control surface

We can navigate, inspect files, run commands, read permissions, and reason about streams.

This week is about precision

The shell rewrites command lines before running them. Quoting, variables, regexes, exit status, and safety flags decide whether automation is reliable or fragile.

Bash is everywhere, and small mistakes matter

Research finding

A study of real open-source Bash scripts found common bugs around quoting, word splitting, path management, command options, return values, and error handling.

- These are not obscure edge cases; they appear in widely used projects.
- The shell rewards careful habits and punishes silent assumptions.
- FIT2109 uses Bash scripts even when your interactive shell is Zsh.

Reference

Bash in the Wild: Language Usage, Code Smells, and Bugs — TOSEM 2022

A short history of Bash

- Unix shells began as command-line interfaces for the operating system.
- The Bourne shell, `sh`, became an early standard Unix shell.
- Bash means **Bourne Again SHell**.
- Bash is both an interactive command interface and a scripting language.
- The same evaluation rules apply when you type commands and when a script runs them.

Practical consequence

Learning Bash is not just learning command names. It is learning how the shell interprets text.

Learning goals

By the end of this workshop, you should be able to:

- Predict how quoting, variables, globs, and splitting affect a command.
- Use `export`, `env`, `$?`, `&&`, and `||`.
- Distinguish shell globs from regular expressions.
- Build `grep`, `find`, `sort`, `uniq`, and `wc` pipelines.
- Review and improve Bash scripts using arguments, conditionals, loops, simple functions, and safe quoting.
- Explain SSH keys, `ssh`, `scp`, and remote command execution.

The shell rewrites before it runs

Shell evaluation model

The shell processes a command line in stages.

1. Read the input line.
2. Parse quoting and structure.
3. Expand variables, globs, and command substitutions.
4. Split unquoted results on whitespace.
5. Execute the resulting command with its arguments.

Key idea

What you type is not always what the command receives.

Quoting controls what the shell changes

No quotes

Variables expand, then split on spaces.

```
file="Week 2.txt"
wc -l $file
```

Shell passes two filenames: `Week` and `2.txt`

Double quotes

Variables expand, but spaces stay inside one argument.

```
wc -l "$file"
```

Shell passes one filename: `Week 2.txt`

Single quotes

Text is literal. No variable expansion.

```
wc -l '$file'
```

Shell looks for a file named: `$file`

Professional habit

Quote variables that may contain spaces or special characters: `"$file"`, `"$1"`.

Variables can stay local or travel

Shell variable

Visible in the current shell.

```
course=FIT2109
echo "$course"
```

Environment variable

Inherited by child processes after `export`.

```
export course
bash -c 'echo $course'
```

Per-command environment

```
COURSE=FIT2109 env | grep '^COURSE='
```

Exit status lets commands make decisions

Key idea

Every command returns an exit status. 0 usually means success; nonzero means failure.

- `$?`: exit status of the previous command.
- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeds.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` fails.

Common script pattern

```
cd "$dir" || exit 1
```

Live demo: shell evaluation

Watch

Compare unquoted and quoted variables, local variables and exported variables, and commands that succeed or fail.

Note

The most important question is not "what command was typed?" It is "what arguments did the command receive?"

Continue: exit status as control flow

Watch

Use `$?`, `&&`, and `||` to make the shell react to success and failure.

Pause and reason

Why should a script stop if `cd "$dir"` fails?

Handout Activity 1: shell evaluation

Now work on Activity 1 in the handout

Predict quoting behaviour, explain variable visibility, and reason about exit status.

Group output

Complete Parts A–D: argument prediction, variable/export reasoning, exit-status decisions, and one safe-command rewrite.

Globs and regexes are different languages

Shell glob

Expanded by the shell before the command runs. Matches filenames.

```
*.log
```

Regular expression

Interpreted by a tool. Matches text.

```
.*\.log$
```

Common mistake

Do not confuse a filename pattern with a text pattern. Quote regexes so the shell does not change them first.

grep flags shape the question

- `-E`: extended regex.
- `-o`: print only the matched text.
- `-n`: show line numbers.
- `-q`: quiet; exit status only.
- `-r`: search recursively.
- `-i`: case-insensitive.

Useful patterns

`grep -E 'WARN(ING)?'` matches `WARN` and `WARNING`.
`grep -Eo 'ERROR [0-9]+'` extracts just the error code.

Text tools compose into pipelines

Key idea

Each command should do one small job, then pass text to the next command.

- `grep`: select matching lines or substrings.
- `sort`: put equal values together.
- `uniq -c`: count adjacent repeated lines.
- `wc -l`: count lines.

Pipeline reasoning

Always explain a pipeline one stage at a time.

`find` searches by file metadata

Key idea

`find` locates files using filesystem properties: name, type, permissions, and full paths.

- `-name '*.log'`: filename pattern. Quote the glob.
- `-type f`: regular files only.
- `! -perm -u=x`: files not executable by owner.
- `-regex`: regex over the full path.
- `-exec cmd {} +`: run a command on matching files.

Live demo: `grep`, pipelines, and `find`

Watch

Start from a concrete question about logs and source files, then build the command piece by piece.

Note

The hard part is choosing the right question: filename search, text search, counting, or metadata filtering.

Pause and reason: full-path matching

Question

Why might a `find -regex` pattern that looks correct fail unless it accounts for the leading path?

Hint

`find -regex` does not only match the final filename.

Handout Activity 2: regex and text tools

Now work on Activity 2 in the handout

Distinguish globs from regexes, design `grep` commands, build a frequency pipeline, and choose `find` predicates.

Group output

Complete Parts A–D: pattern classification, grep design, pipeline explanation, and find command design.

Scripts make workflows reusable

Key idea

A shell script is a text file that Bash reads and evaluates, just as if you typed each command yourself.

- Shebang: `#!/usr/bin/env bash`
- Positional arguments: `$1`, `$2`, ...
- Argument count: `$#`
- Default value: `${1:-world}`
- Conditional: `if`, `then`, `else`, `fi`
- Loop: `for x in ...; do ...; done`

Bash can group commands into functions

Recognition pattern

A Bash function names a small command sequence so the script can call it more than once.

```
warn() {
  echo "warning: $1" >&2
}

warn "missing input"
```

Week 2 boundary

Today you only need to recognise this pattern. We still focus on arguments, `if`, loops, quoting, and exit status.

Reliable scripts fail clearly

Three safety habits

- `set -u`: fail on unset variables.
- `set -o pipefail`: fail if any pipeline stage fails.
- Quote variables: `"$1"`, `"$file"`.

Why this matters

Silent empty strings and hidden pipeline failures can make a script appear successful when it did the wrong thing.

Live demo: repair a small script

Watch

Start from a fragile script, then add safe quoting, failure messages, exit status, and safety flags.

Note

A safer script is easier to explain: what input it expects, what failure means, and what evidence proves success.

Continue: loop over log files

Watch

Turn a one-file command into a reusable loop over many log files.

Pause and reason

Where should the loop handle the "no match" case: before the loop, inside the loop, or after the loop?

Handout Activity 3: script review and repair

Now work on Activity 3 in the handout

Review a fragile script, identify failure modes, and rewrite it using safer Bash habits.

Group output

Complete Parts A–D: bug list, safer rewrite, exit-status explanation, and loop design.

SSH connects local workflows to remote machines

Key idea

SSH encrypts a connection between machines. It lets you log in, run remote commands, and move files.

- `ssh user@host`: interactive login.
- `ssh user@host 'command'`: run one remote command.
- `scp local user@host:/path/`: copy to remote.
- `scp user@host:/path/file .`: copy from remote.

SSH keys separate public and private trust

Private key

Stays on your machine. Never share it.
Protect it like a password.

Public key

Copied to the remote server, usually into `~/.ssh/authorized_keys`.

Mental model

The server grants access when your private key matches a public key it trusts.

Live demo: remote workflow

Watch

Read the shape of an SSH workflow: generate or use a key pair, connect, run one command remotely, and transfer a file.

Note

The applied class should not become an SSH installation debugging session. Focus on the workflow and evidence.

Handout Activity 4: SSH workflow planning

Now work on Activity 4 in the handout

Plan a remote workflow using `ssh`, `scp`, public/private keys, and remote command execution.

Group output

Complete Parts A–D: key explanation, command sequence, failure diagnosis, and reusable workflow sketch.

Final checkpoint: Poll Everywhere

pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to pollev.com/fit2109.

Purpose

Use the Poll Everywhere questions to check which Week 2 ideas need one more example before we finish.

What should feel different after today?

- You can explain why quoting changes command behaviour.
- You can use exit status to make shell workflows branch safely.
- You can distinguish globs, regexes, text search, and file search.
- You can build a pipeline to answer a question from text.
- You can review a script for quoting, missing input, and hidden failures.
- You can describe how SSH keys support remote command-line workflows.

Q&A and next steps

Questions?

Before the applied class

Practise explaining not just what a command does, but why it is written that way.